

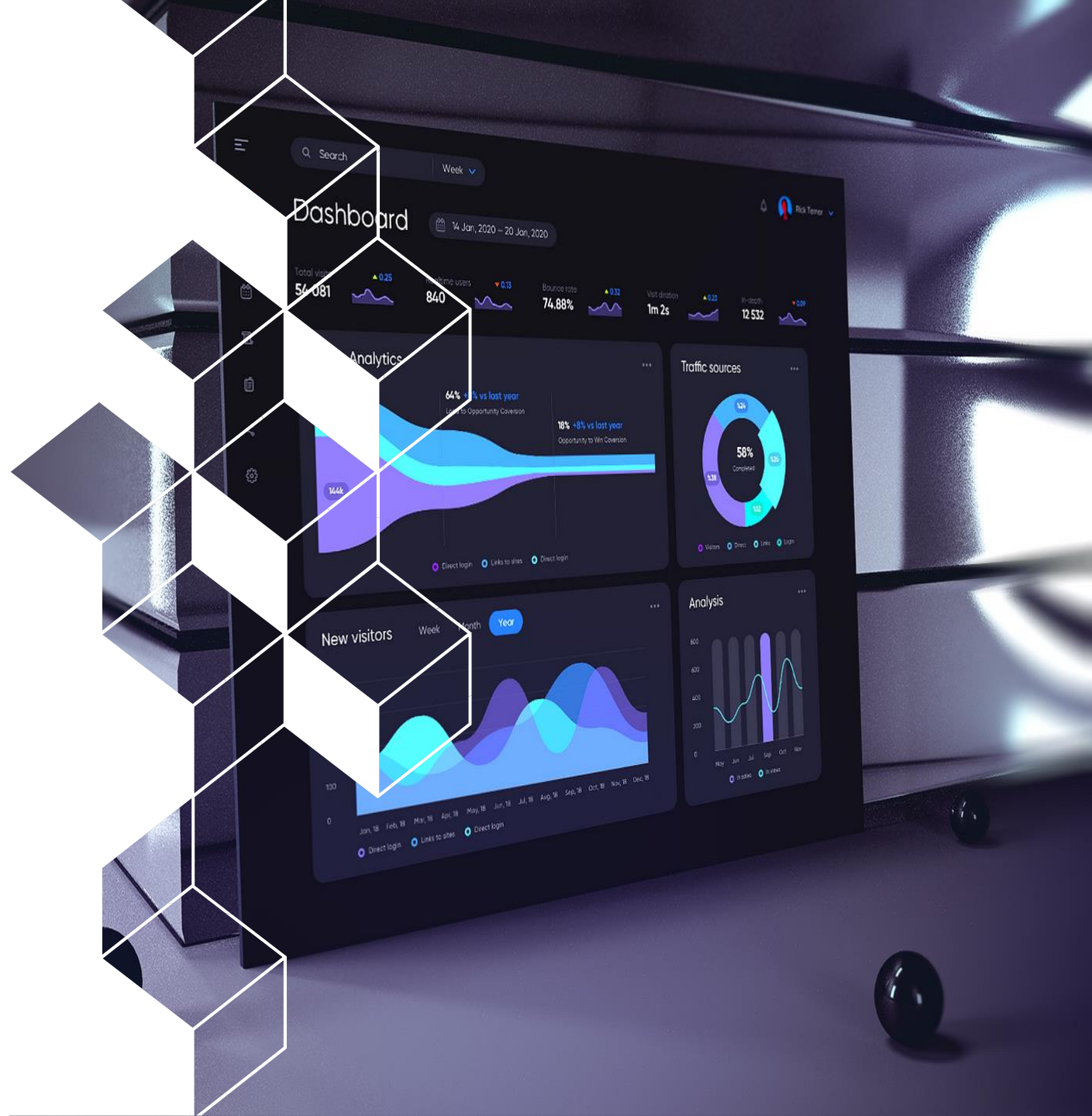


GUI Strategies Workshop

Katy SAINTIN & Paul LOTRUS
IRFU, CEA, Université Paris-Saclay
Gif-sur-Yvette, France



katy.saintin@cea.fr



My experience in GUI development

1998 – 2001 : 

- LLB spectrometers at CEA of Saclay
- C++ developer on High Level Application - ILOG Views
- A new generation of software control spectrometers – NOBUGS 2000



2004 – 2017 :

- SOLEIL synchrotron – Beamlines & Accelerator - 
- Java developer on High Level Application – Swing and COOX
- COMETE: A Multi Data Source Oriented Graphical Framework – ICALEPCS 2011



Since 2017 :

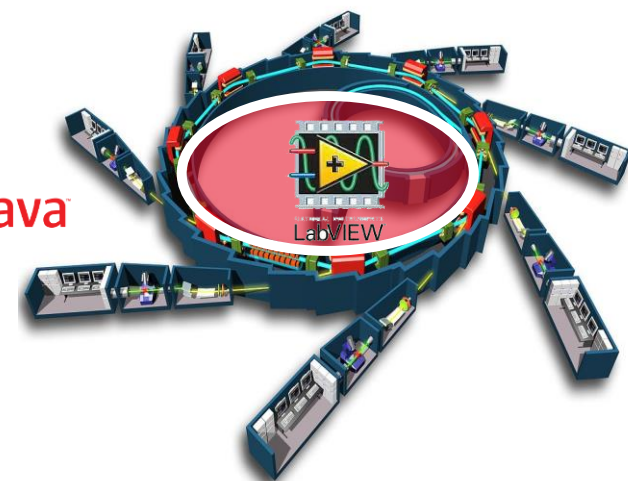
- IRFU CEA Saclay – MUSCADE & EPICS
- Java developer on High Level Application – Swing and CS-Studio
- Data Visualization With Data Browser Software – ICALEPCS 2019



What technologies and for what ?

TANGO @ SOLEIL :

- Java rich client application(ATKPanel, Jive, [DataBrowser](#)) by IT team
- [COOX](#) SCADA in Java for beamline, by beamline staff
- LabVIEW for accelerator supervision, by operators

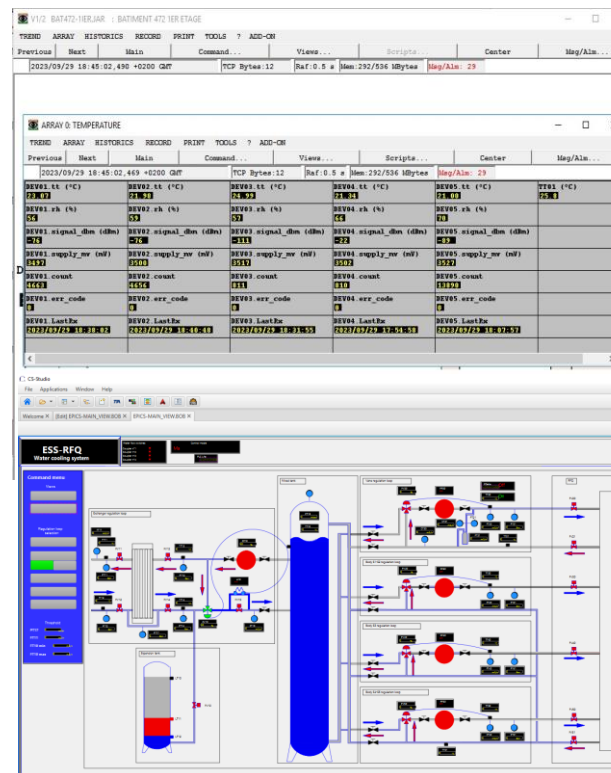


MUSCADE @ IRFU :



- Java rich client application (ANIBUS) and
- Android application for Smartphone
- Java Web Start

EPICS @ IRFU :



What issues with these technologies?



Framework	Pros	Cons
TANGO	➤ QT, Python, Java, LabView ➤ Color code is fixed ➤ Object Oriented, common interface ➤ Device database => naming and dynamics GUI ➤ Documentation ➤ Interoperability COOX ➤ Common Layer ➤ Reusable widget ➤ Drag & Drop technology ➤ Java portability ➤ Security management	➤ No web solution widely used by all institutes ➤ COOX (not free) (but JDraw is free) ➤ No common strategy between institutes ➤ No security management on TANGO ➤ GUI only dedicated to TANGO devices
MUSCADE	ANIBUS ➤ Optimized for small experiences ➤ Adapted to PLC Programmer needs ➤ Java portability ➤ Security management (certificates) ➤ Android application	➤ Support java only ➤ Java Web Start not supported anymore ➤ Not user friendly => AutoCAD ➤ AWT GUI is old fashion look and feel ➤ In-house solution on all stacks (No developer, no support anymore)
EPICS	➤ QT, Python, Java, LabVIEW ➤ DBWR ➤ Archiver Appliance ➤ Security management with gateway CS-STUDIO ➤ Open source ➤ Widely used ➤ Embedded Display ➤ Based on known technologies ➤ <i>Java portability (with issues sometimes Java FX)</i> ➤ Several data sources EPICS, TANGO, MQTT ...	➤ No web solution widely used ➤ No common strategy between institutes (color code) ➤ Channel Access => a PV is read or write ➤ CS-STUDIO has no security management ➤ Documentation is not clear and complete ➤ No dynamic GUI ➤ No PV index (Channel Finder have to be filled) ➤ A PV can be defined several times on the same network ➤ OS dependent due to Java FX

What is our GUI strategy ?

Technical criteria :

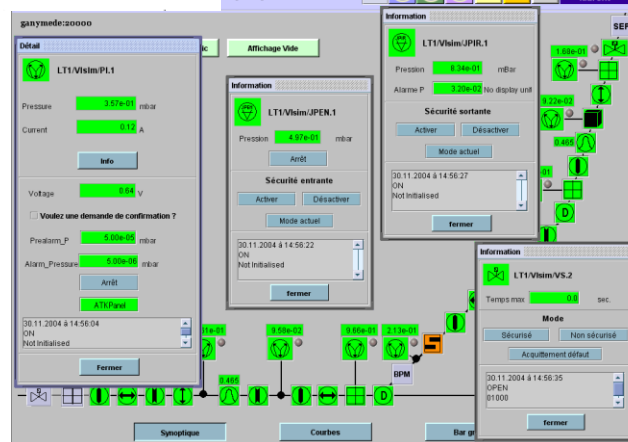
- Portability and interoperability
- User autonomy (Drag and Drop technology)
- Modular and MVC design pattern
- Reusable views
- Web client without software installation (not Java Web Start)

GUI guideline : Document defined at SOLEIL and at IRFU ([see guideline for CEA](#))

- Common layer definition
- Code color fix (TANGO CS it is fixed by the framework , for EPICS color.def configuration file for CS-STUDIO)
- LSAP (Less Smart As Possible) , all the business and logical should be managed by server. (Avoid script in the client)
- Name the widget according to its function . Ex : Textfield_VoltageValue for a Textfield which displays a voltage value
- Reuse embedded display and defined MACRO or parameters

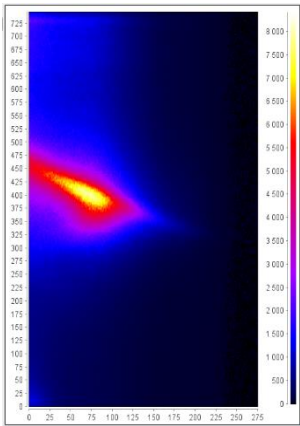
Future development :

- Alarm management web client for EPICS
- Security implementation in CS-Studio
- PV Access IOC Migration => avoid Read and Write PV

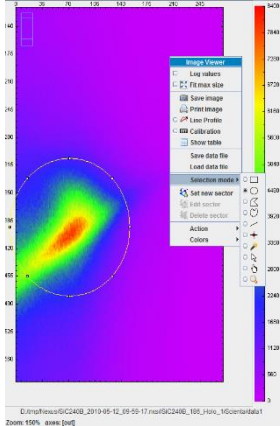
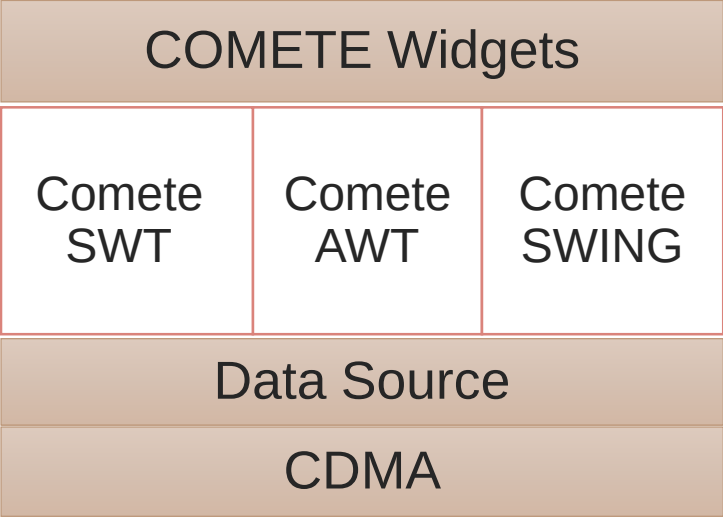


Screenshots (COMETE MVC design pattern)

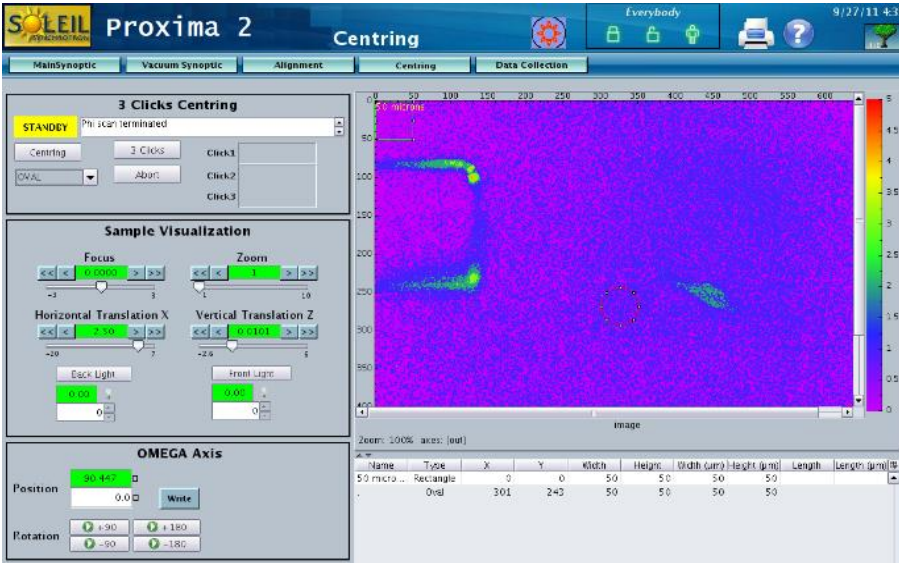
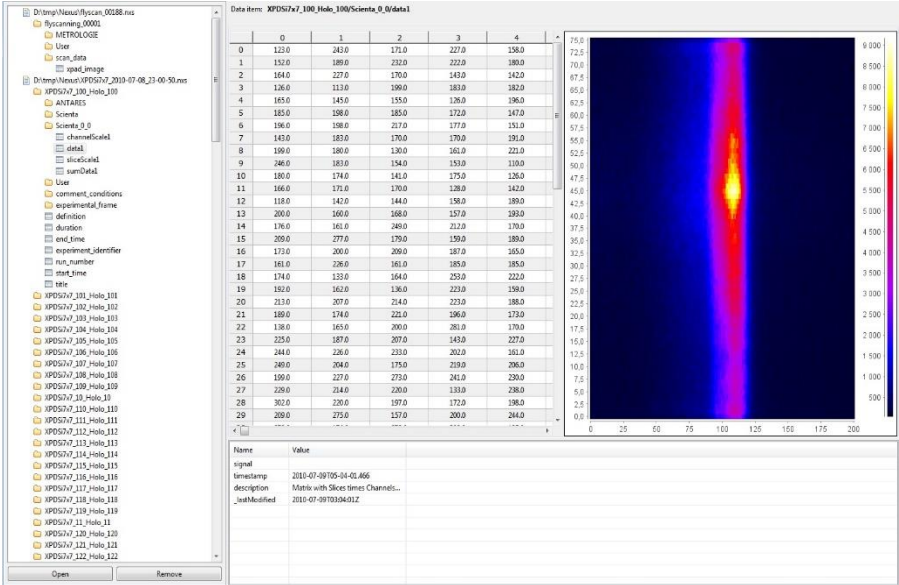
COMmunity of Extensible Toolkit for Experiment



ImageViewer SWT

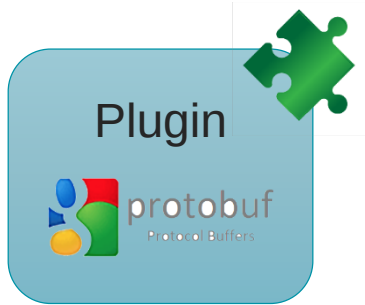
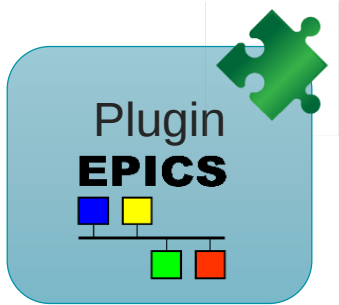
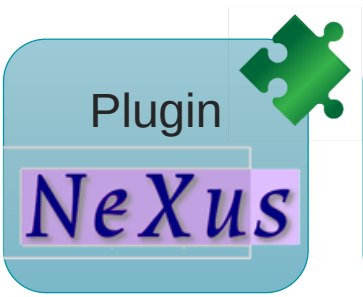
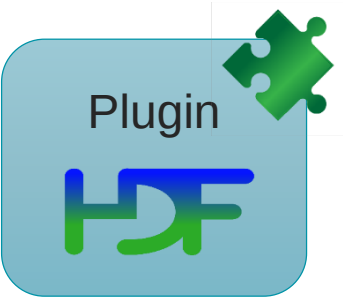


ImageViewer SWING



CDMA Framework (Data source abstraction)

Common Data Model





irfu

Any questions ?

Thank you for your attention